

Algorithms: Ray Tracing

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory
Department of Electrical and Computer Engineering
Cullen College of Engineering
University of Houston

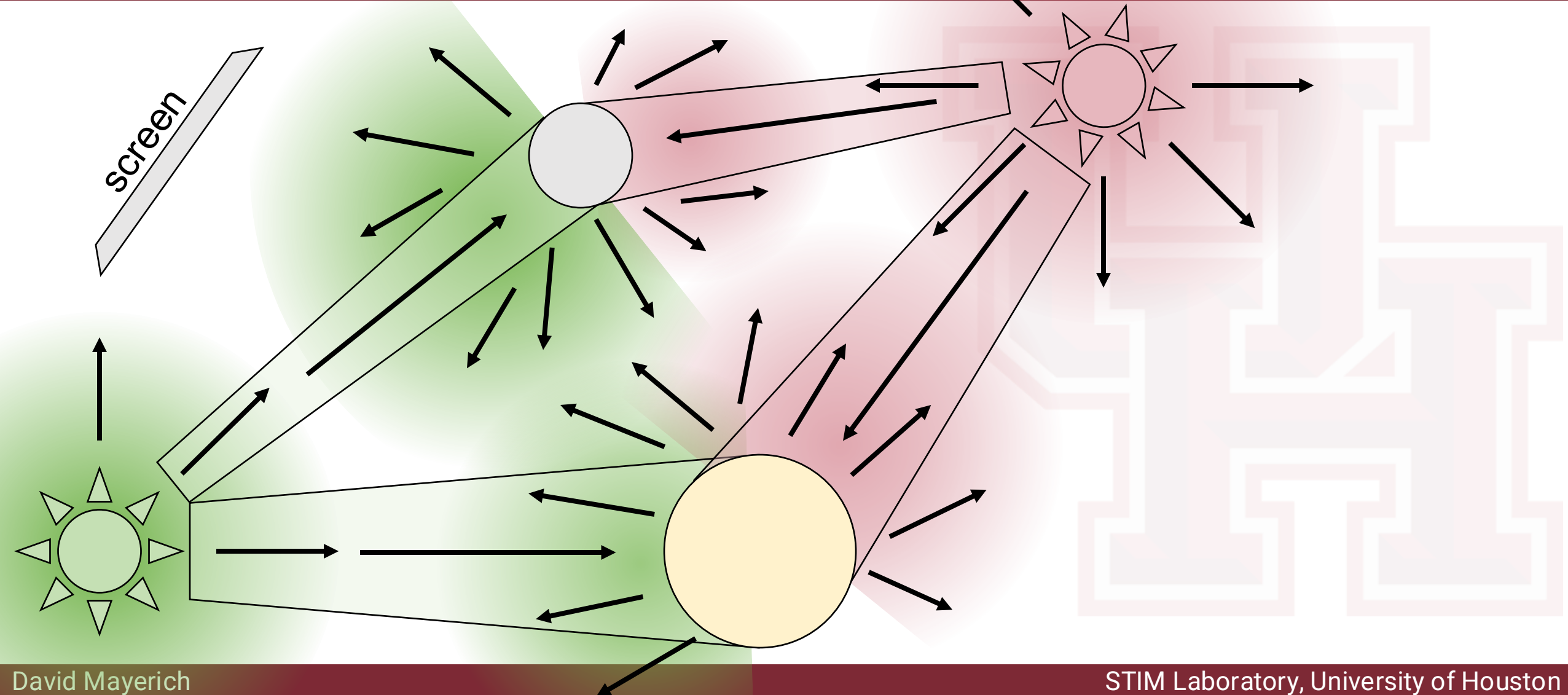
UNIVERSITY of **HOUSTON** | ECE

Ray Tracing: Goals

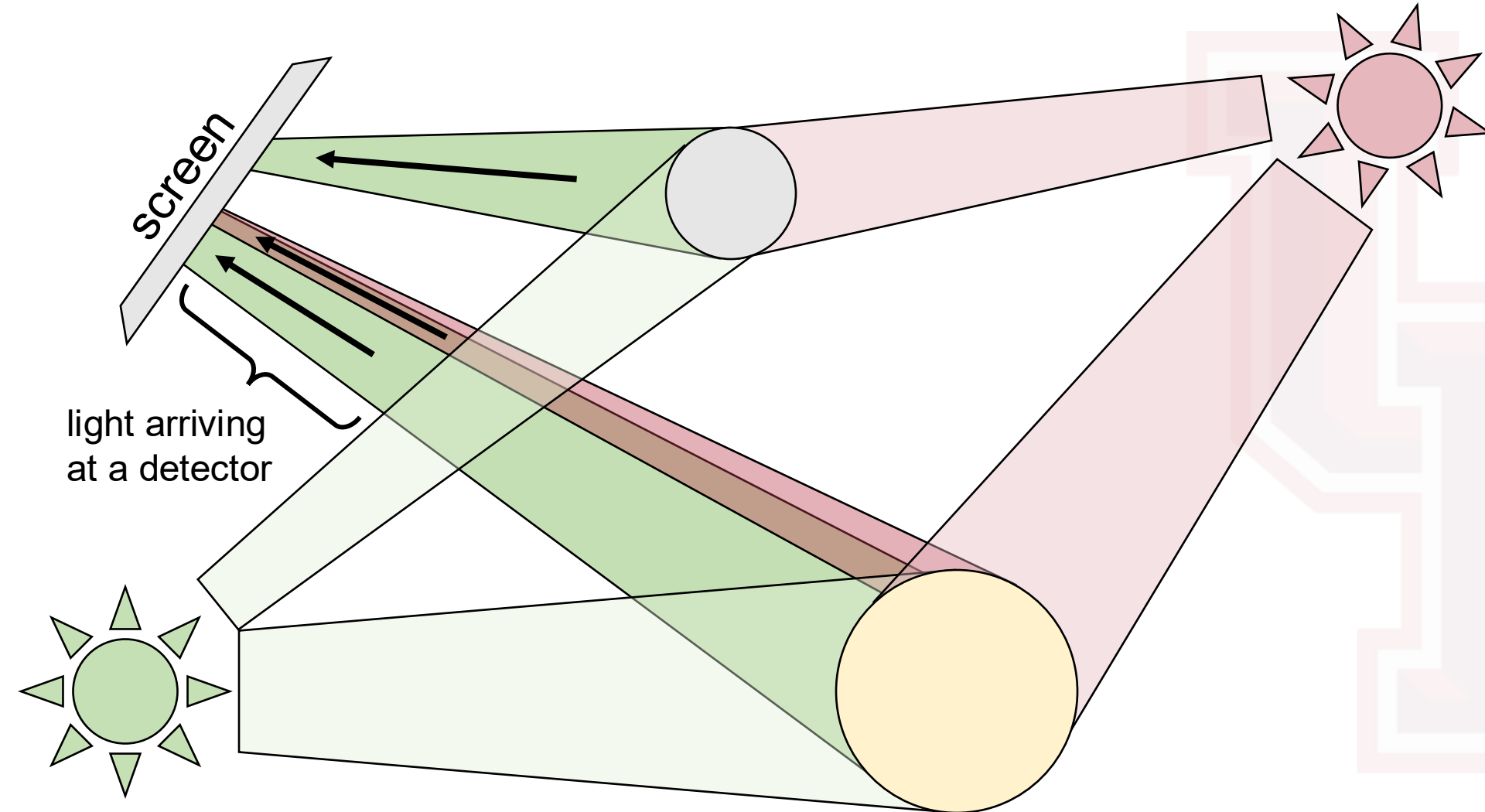
- Category of algorithms used to synthesize digital images
- Determine pixel color by back-tracking the light path
- Produces realistic images of complex materials
- Computationally expensive – computing resources increase dramatically with image complexity
- Potentially very parallel: each pixel in the image is independent



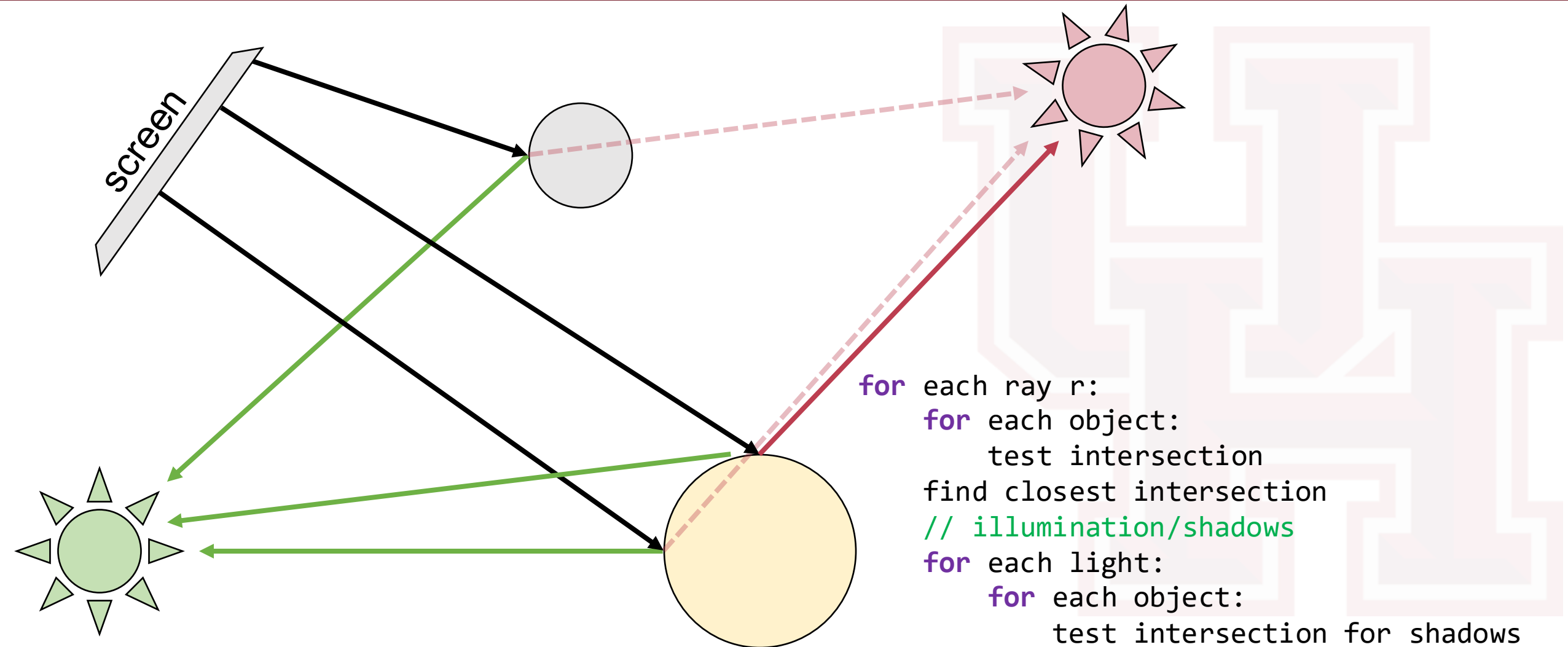
Light Scattering: “Forward Model”



Light Scattering: “Forward Model”



Ray Tracing: “Inverse Model”

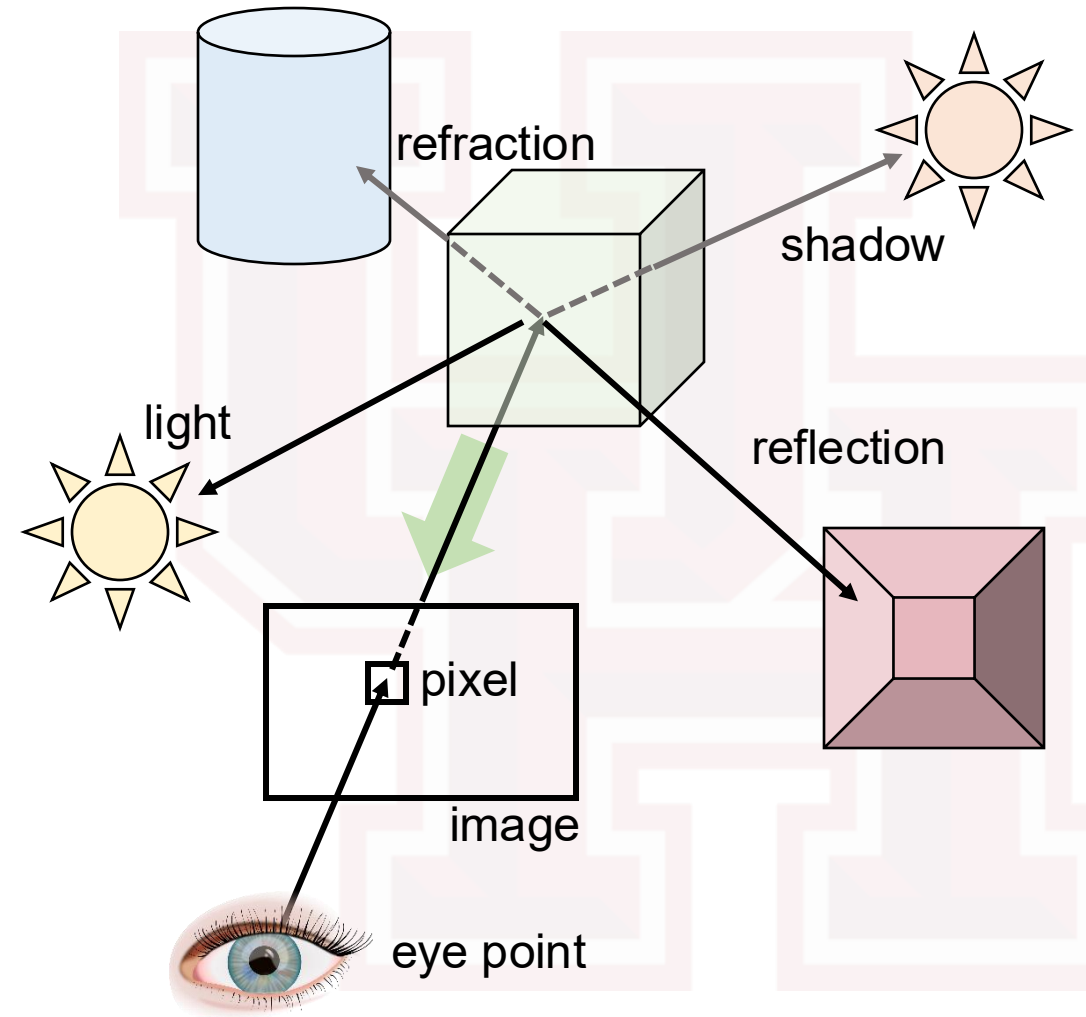


Ray Tracing Algorithm

for each pixel:
 find all ray intersections
 calculate closest intersection
 compute color at the intersection

$$C_i = C_e + C_d + k_r C_r + k_t C_t$$

- C_i is the light at the intersected object
- C_e is the emission from the object
- C_d is the direct illumination from incident light
- C_r is the light reflected from the object
- C_t is the light transmitted from the object
- k_r and k_t are material properties for reflection and transmission



Recursive Ray Tracing

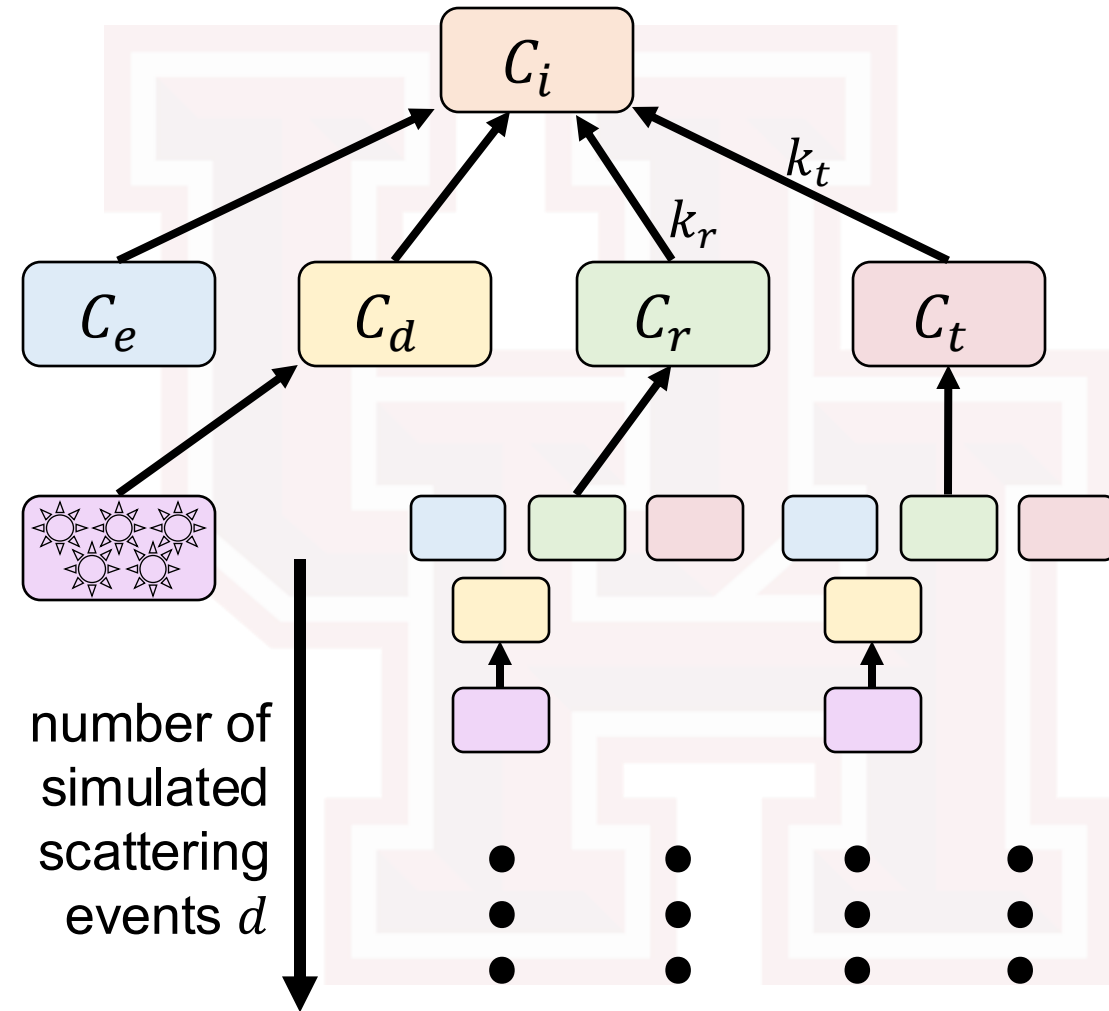
$$C_i = C_e + C_d + k_r C_r + k_t C_t$$

- Assume N objects, L lights, and P rays
 - Detecting the nearest object is $O(N)$
 - C_d tests intersections between lights and objects: $O(NL)$
 - C_r and C_t require recursion:

$$O(P[N + NL + R + T])$$

$$\text{where } R = T = N + NL + R' + T'$$

$$O(2^{d+1}[N + NL])$$



Recursive Ray Tracing

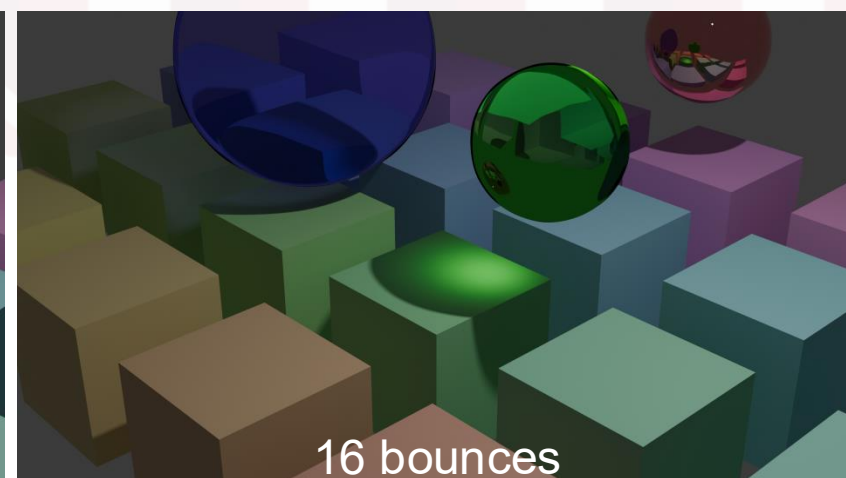
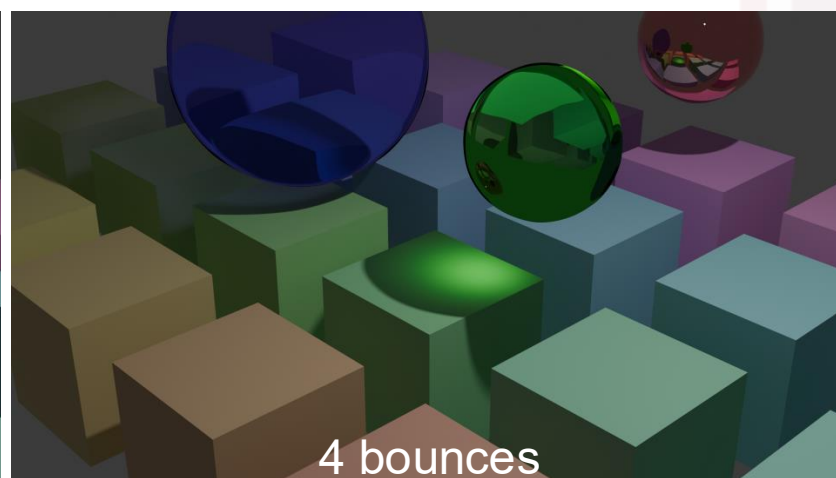
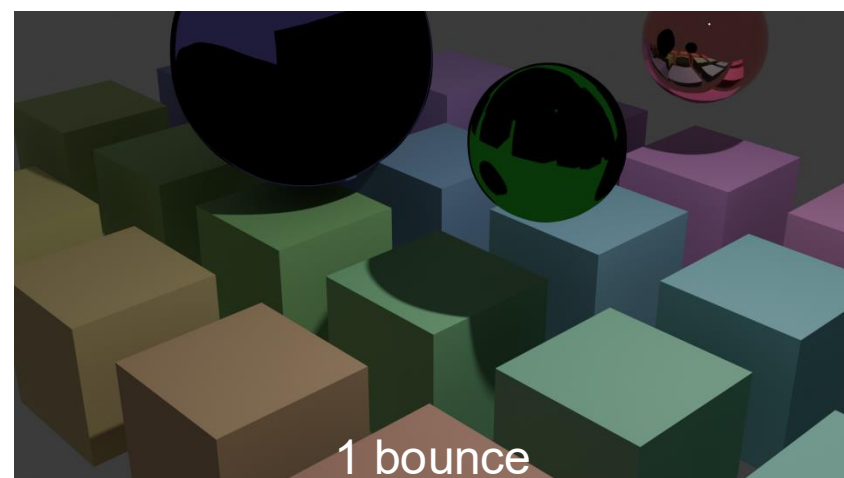
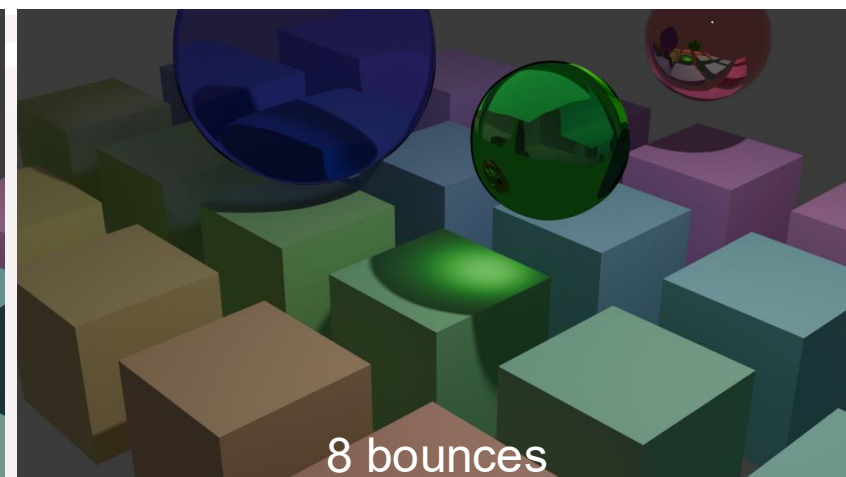
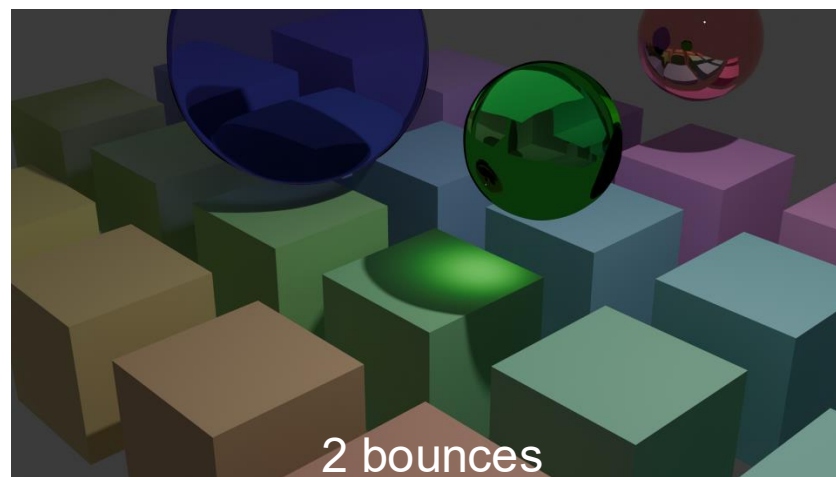
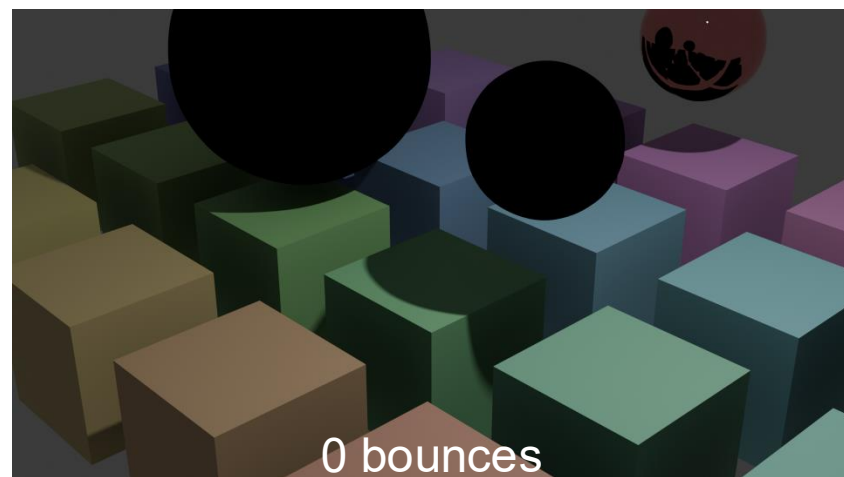
```
for each pixel ray r:  
    color pixel = LaunchRay(r, 0)  
  
LaunchRay(r, b++):  
    if(b > BOUNCE_MAX) return black;  
    for each object: test intersection  
    for each light: l_color += ShadowRay(light)  
    r_color = LaunchRay(reflection, b++)  
    t_color = LaunchRay(transmission, b++)  
    return l_color + r_color + t_color
```

```
ShadowRay(r):  
    for each object:  
        if intersection, return false  
    return true
```



Obligatory ray-tracing example by Gilles Tran, oyonale.com
POVray (povray.org)

Recursive Ray Tracing



*rendered using Blender with Cycles ray tracing

Recursive Ray Tracing

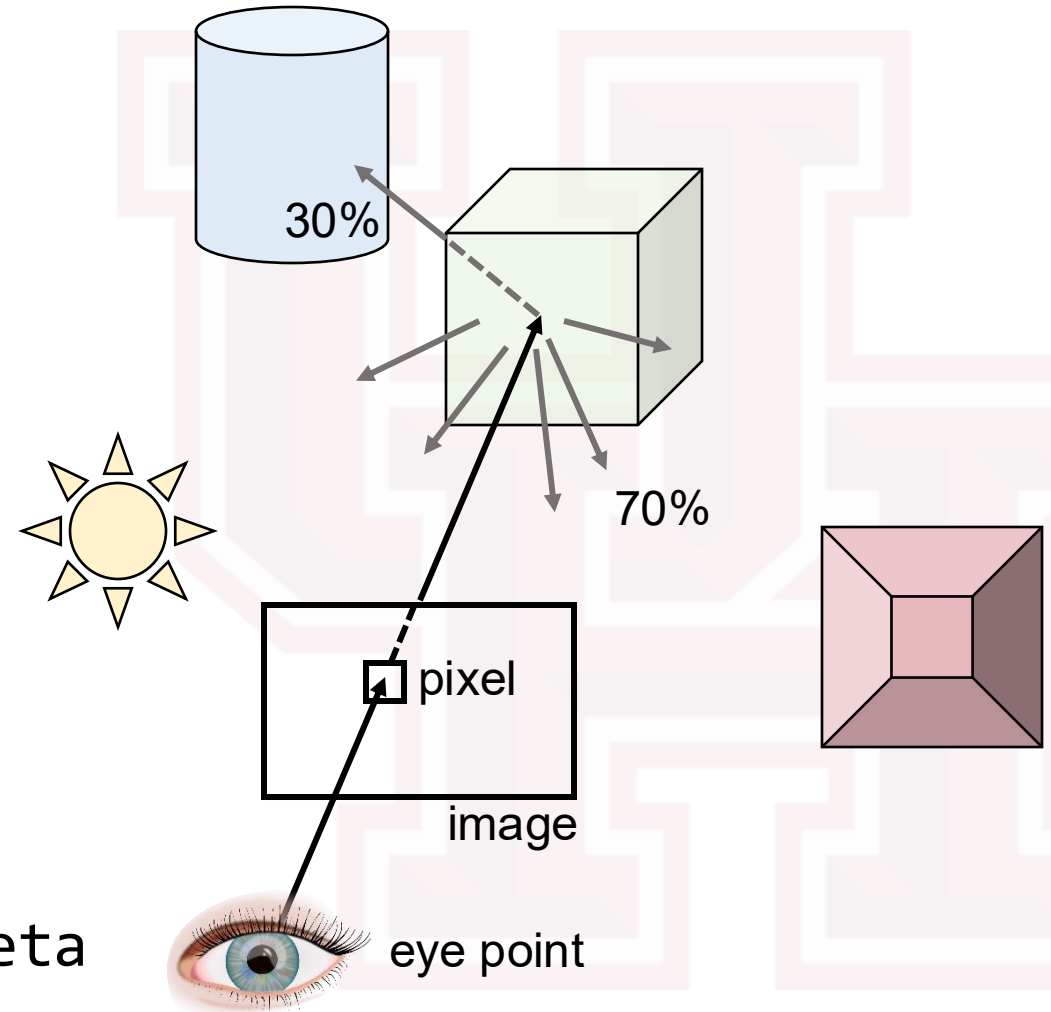
*rendered using Blender with Cycles ray tracing



Path Tracing

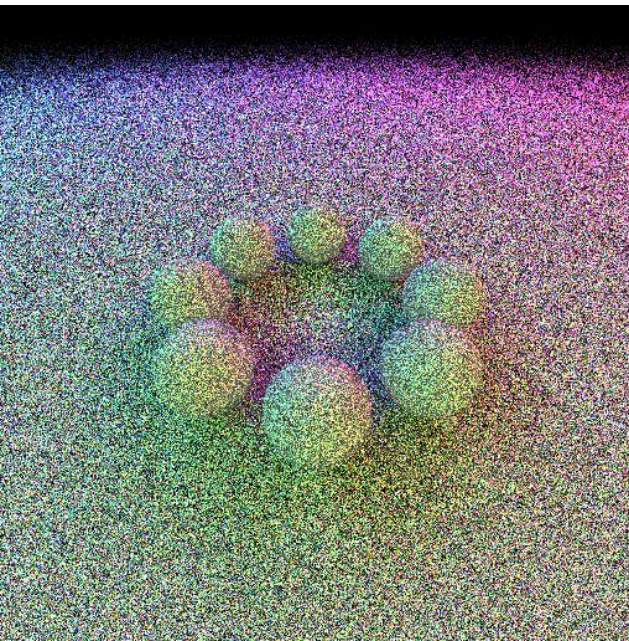
- Statistically sample individual ray paths
- ```
for each pixel ray r:
 color pixel = LaunchRay(r, 0)
```

```
LaunchRay(r, b):
 if(b > BOUNCE_MAX) return black;
 entity = find_nearest_entity(r)
 color_emit = entity.emission()
 next_ray = entity.scatter()
 cos_angle = cosine(r, next_ray)
 color_in = LaunchRay(next_ray, b++)
 return color_emit + color_in * cos_theta
```

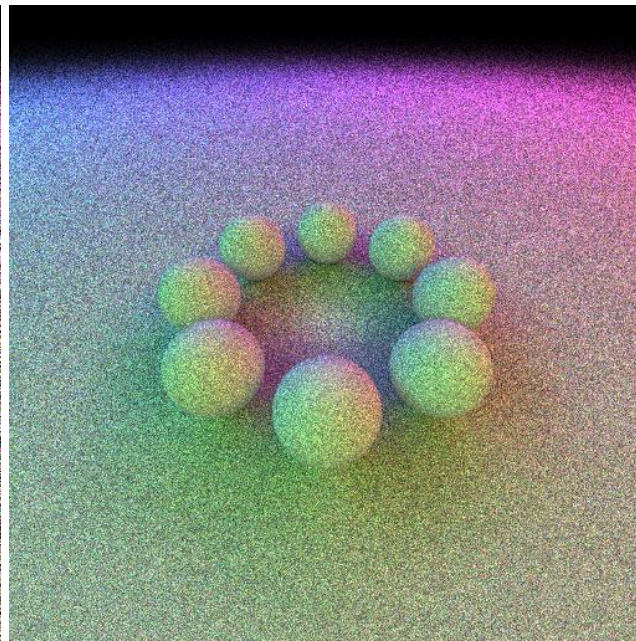


# Path Tracing

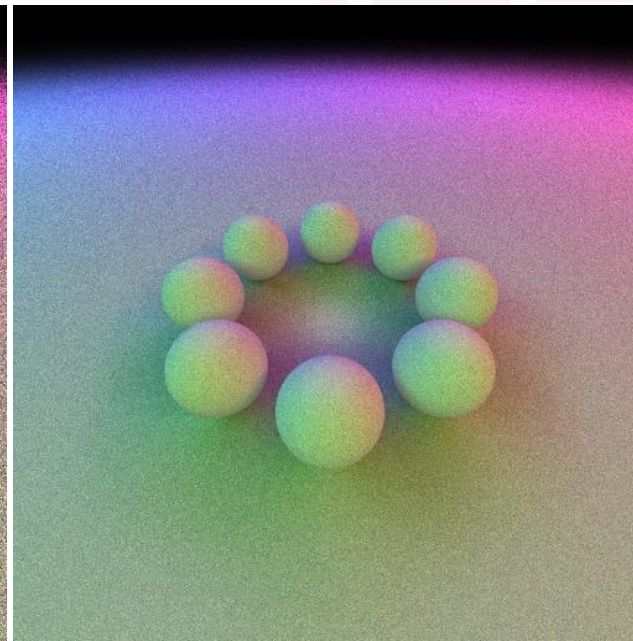
- Average several paths instead of one “deep” recursive tree
- Trade evaluation time for noise
- Solve using Monte-Carlo integration



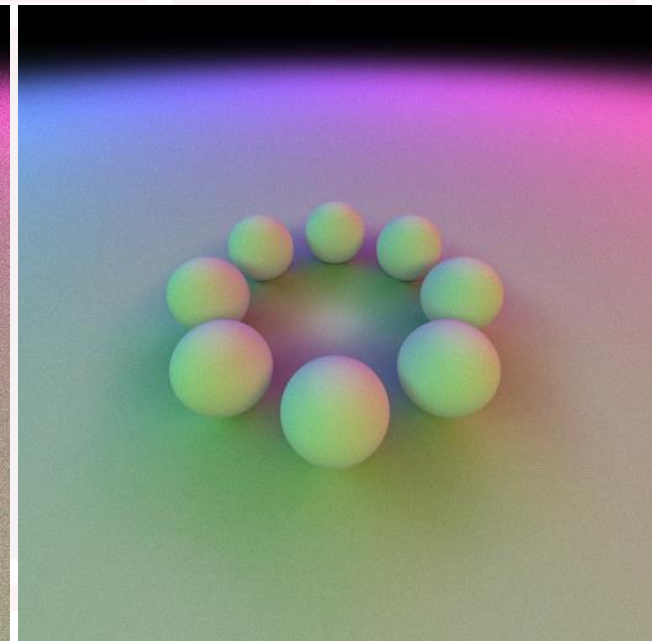
100 samples



1,000 samples



10,000 samples



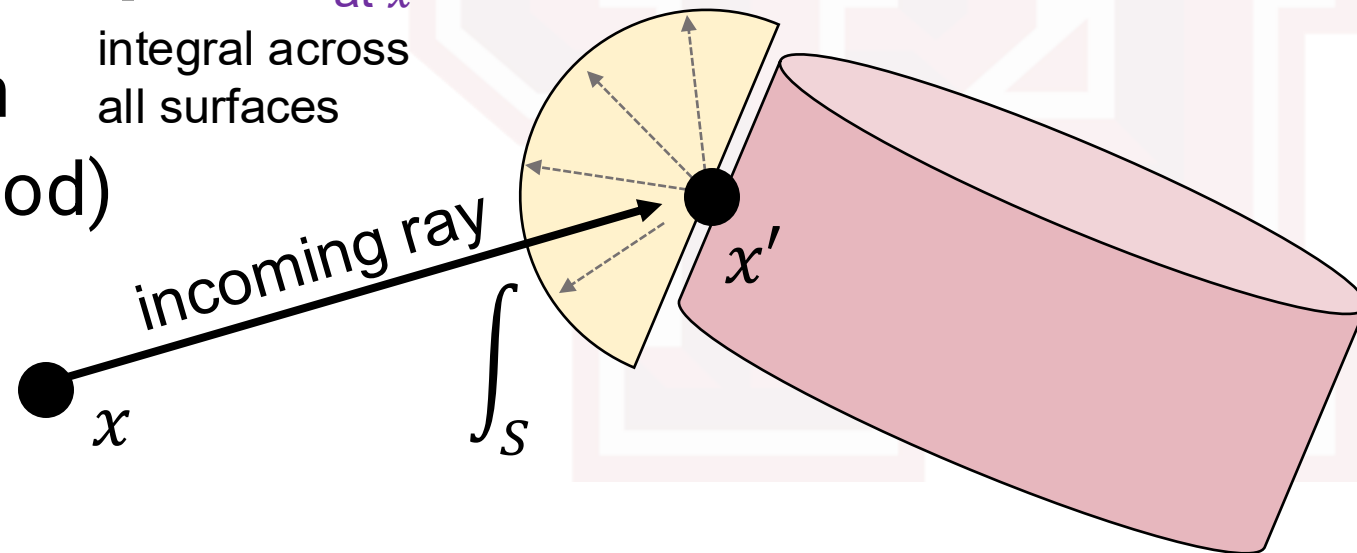
100,000 samples

# Rendering Equation

$$\underbrace{I(x, x')}_{\text{light intensity from } x' \text{ to } x} = \underbrace{g(x, x')}_{\text{geometry term}} \left[ \underbrace{\epsilon(x, x')}_{\text{emitted light from } x' \text{ to } x} + \int_S \underbrace{p(x, x', x'')}_{\text{intensity of light scattered from } x'' \text{ to } x \text{ by the patch at } x'} \underbrace{I(x', x'')}_{\text{light intensity from } x' \text{ to } x} dx'' \right]$$

↑  
integral across all surfaces

- Approximates Maxwell's Equation
- Geometric optics (Luneburg Method)
- Solve the integral by:
  - Recursing through  $I(a, b)$
  - Use Monte-Carlo integration

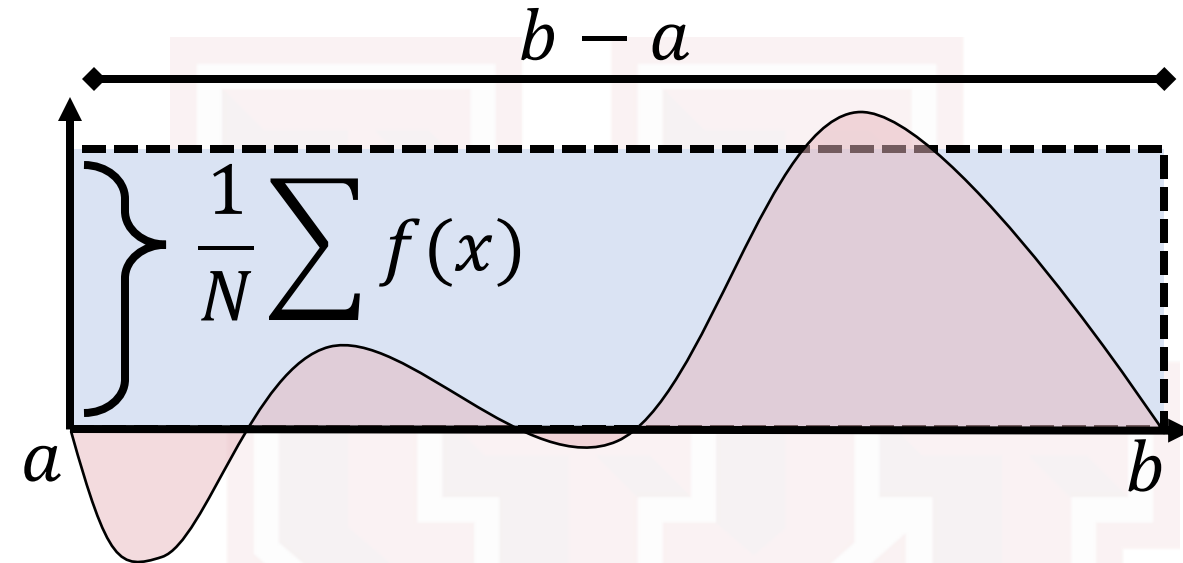


# Monte-Carlo Integration

- Numerically solve an integral using random numbers:

$$\int_a^b f(x) = \frac{b-a}{N} \sum_{i=0}^{N-1} f(x_i)$$

- The integral of a function can be calculated from its mean
- Approximate the mean using random sampling



- Advantage: The error of the approximation decreases with:

$$\epsilon \approx \frac{1}{\sqrt{N}}$$

(independent of the dimension)